

Теоретические основы информатики

Разделы	Темы
Информация и данные	Основные понятия информатики Кодирование информации
Двоичная математика	Системы счисления Битовые операции
Модели и структуры данных	Типы данных Структуры данных
Хранение и обработка информации	Структурированные файлы Парсинг данных Регулярные выражения
Методы информатики	Алгоритмизация Сортировка

Беляков Андрей Юрьевич

к.т.н., доцент

Тема 5

gitverse.ru/band/TOI

Раздел 3. Модели и структуры данных

Тема 5

Типы данных

- Понятие типа данных.
- Роль типа данных: множество допустимых значений и операции.
- Простые и составные типы данных.
- Структуры (записи) .
- Пользовательские типы данных.

Тема 6

Структуры данных

- Абстрактные структуры данных: стек, очередь, множество, словарь, дерево.
- Интерфейс абстрактных структур данных.
- Реализация алгоритмов с помощью абстрактных структур данных.

Тема 5

Типы данных



Вопрос 1: Модели представления данных

Вопрос 2: Особенности типов данных

- переходим от байтов на более высокий уровень абстракции
- в памяти нет разницы между числом **65** и символом **'А'**
- эту разницу может увидеть программист или программа
- эта разница задаётся **типом данных**
- а как описывать предметную область – **модель данных**

Вопрос 1

Модели представления данных

Модель данных – это концептуальное описание того,
как данные организованы,
как они связаны между собой и
какие операции над ними допустимы

Модель данных не зависит от конкретной реализации на
каком-либо языке программирования (тип данных зависит)

-> **как описать (декларировать) :**

точку и круг

книгу и библиотеку

студента и рейтинг студентов

сеть дорог

иерархию наследования

Модели данных

Иерархическая

Сетевая

Реляционная

Объектная

Документная

Графовая

Модель данных не привязана к языку программирования

Тип данных зависит от конкретной реализации на каком-либо языке программирования

Модели данных

иерархическая



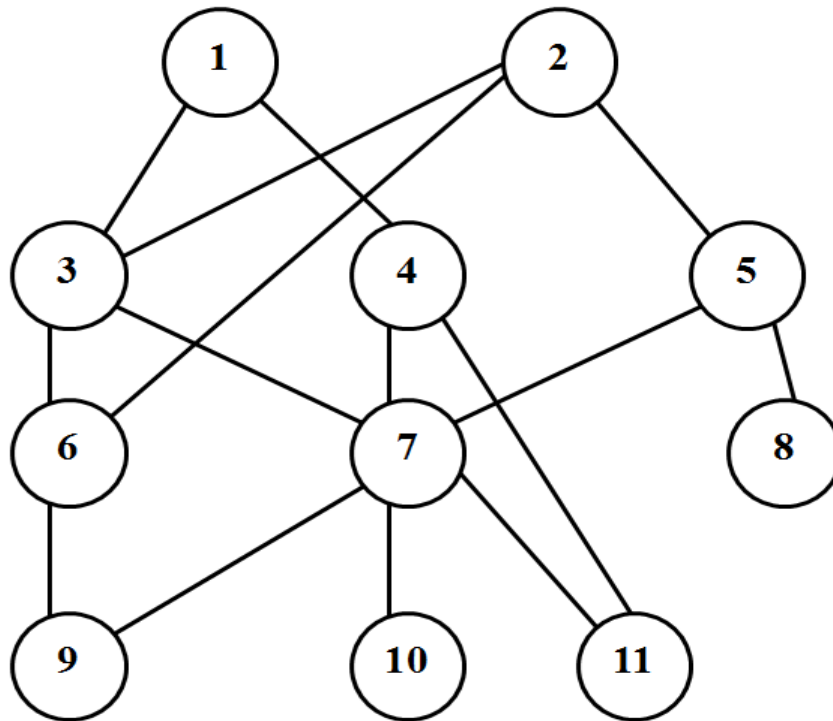
Файловые системы – папки и файлы
XML, HTML – документы
Организационные структуры – армия,
университет

Проблемы:

- Привязка жесткая
- Как показать что преподаватель может работать и на другой кафедре
- Как показать, что студент пишет курсовые и на другой кафедре

Модели данных

сетевая



- это расширение иерархической модели

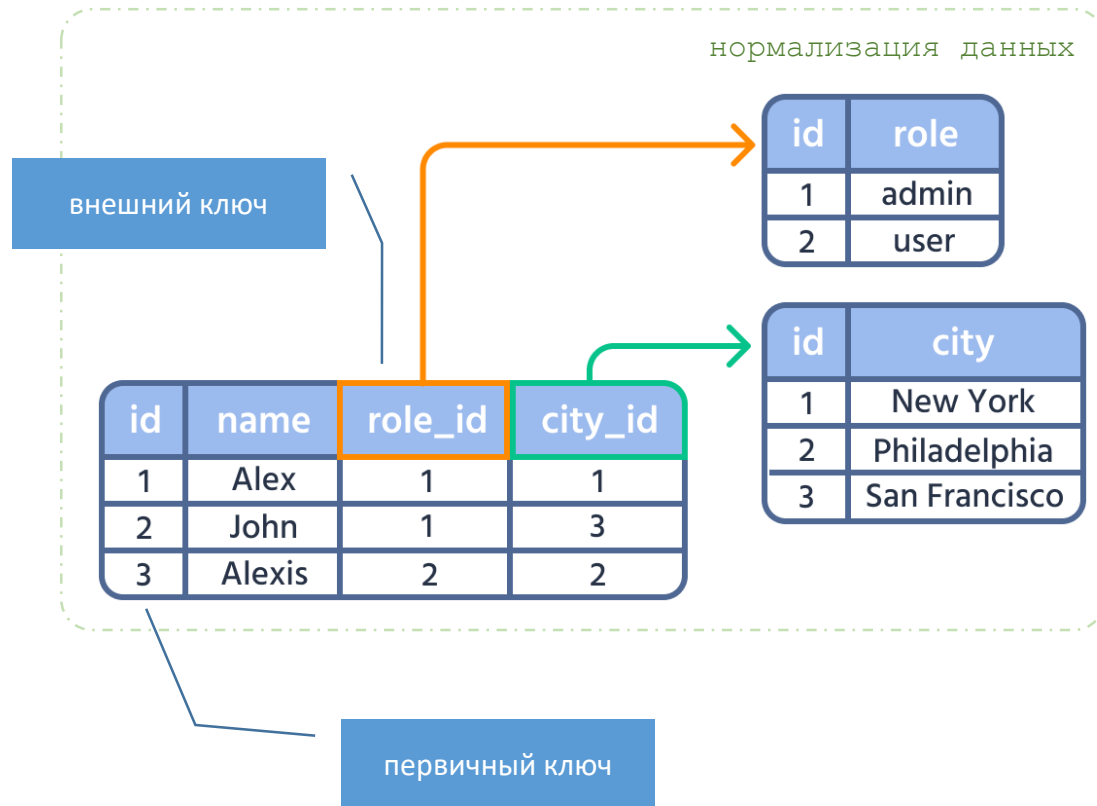
Данные – это список наборов, где набор определяет связь между одним владельцем и множеством записей-членов. Одна запись может быть членом нескольких наборов.

Для получения нужных данных программа должна пройти по сети связей.

+ гибкость представления данных
модель позволяет моделировать сложные взаимосвязи между объектами
преимущество перед иерархической моделью

Модели данных

реляционная



- данные в виде набора связанных таблиц
- основана на принципах теории множеств
- предложил британский учёный Эдгар Кодд 1969

таблица – задаёт отношение и хранит сущности
столбец задаёт свойство сущности – атрибут
домен – набор допустимых значений для атрибута

для манипулирования данными используются специальные операторы

набор ограничений определяет согласованность данных

абстрагирование
модель объекта

Модели данных

объектная

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age  # приватность - инкапсуляция
    def __str__(self):
        return f"{self.name}, {self.__age} лет"
```

```
class Student(Person):  # наследование
    def __init__(self, name, age, group):
        super().__init__(name, age)
        self.group = group
        self.courses = []
    def add_course(self, course):
        self.courses.append(course)
```

```
class Professor(Person):  # наследование
    def __init__(self, name, age, department):
        super().__init__(name, age)
        self.department = department
        self.courses = []
    def teach(self, course):
        self.courses.append(course)
    def __str__(self):  # полиморфизм
        return f"Профессор {self.name}"
```

- данные и методы хранятся вместе
- наследование сокращает код
- контролируемый доступ к полям объектов
- полиморфизм

```
ivanov = Student("Иванов", 20, "ИСУ-01")
smirov = Professor("Смирнов", 45, "МО")
ivanov.add_course("Базы данных")
smirov.teach("Математический анализ")
```

```
persons = [ivanov, smirov]
for person in persons:
    print(person)  # полиморфизм - единый интерфейс
    # Иванов, 20 лет
    # Профессор Смирнов
```

Модели данных

документная

```
{
  "university": "hse",
  "students": [
    {
      "name": "Иванов",
      "age": 20,
      "courses": ["БД", "Программирование"]
    },
    { "name": "Петров", "age": 21 }
  ]
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<book id="bk101">
  <title>Война и мир</title>
  <author>Лев Толстой</author>
  <year>1869</year>
  <price currency="RUB">250</price>
</book>
```

- данные представлены иерархически (JSON, XML)
- не регламентирован уровень вложенности
- документы могут содержать вложенные структуры (документы, массивы)
- отдельный документ – это файл (строка) с вложенной структурой
- документы могут иметь разную структуру
- легко менять структуру
- близка к объектной модели – удобно использовать в программировании

Контрольные вопросы

Модель данных - понятие

Особенности модели данных:

Иерархическая

Сетевая

Реляционная

Объектная

Документная

Графовая



Вопрос 2

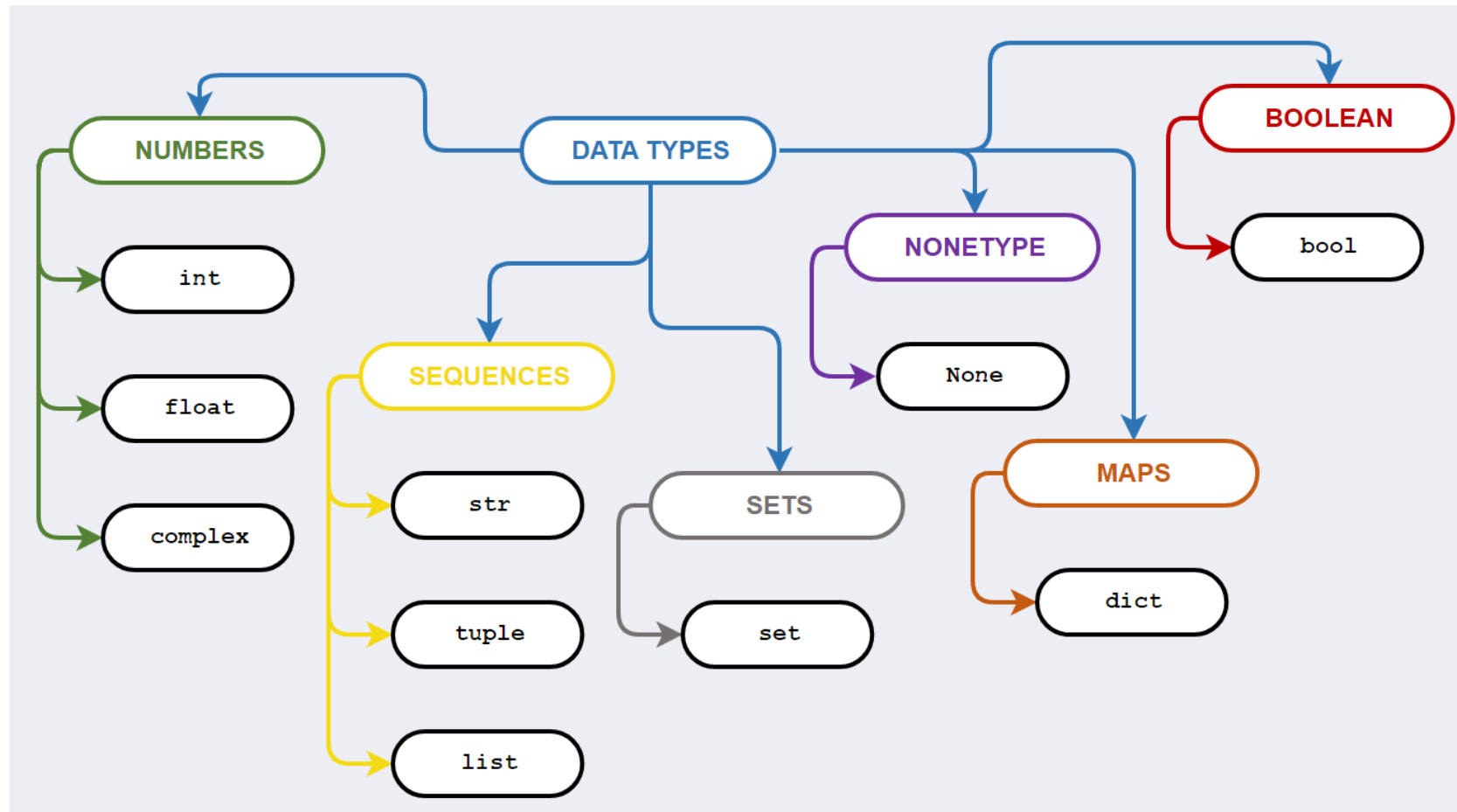
Особенности типов данных



Тип данных — это декларация о:

- множестве допустимых значений
- множестве допустимых операций над этими значениями

Встроенные типы данных



Python

ТОЛЬКО ОСНОВНЫЕ

какая это
модель ?

Явная и неявная типизация

Явная типизация

C, C#, Java, C++

декларация:

```
int num = 5;
```

Проверка типов - на этапе компиляции

Неявная типизация

JavaScript, Python, php

декларация:

```
num = 5
```

Проверка типов - на этапе выполнения

Сильная типизация:

Java, Python, C#

требуется явное преобразование

Слабая типизация:

JavaScript, C

неявное приведение

Python

язык с динамической типизацией, но со строгой (сильной) типизацией - нельзя сложить строку '5' и число 3 без явного преобразования

```
>>> '5' + 3
Traceback (most recent call last):
  File "<python-input-14>", line 1, in <module>
    '5' + 3
    ~~~~^~~~
TypeError: can only concatenate str (not "int") to str
```

- для сравнения язык C:

```
int x = 5 + 3.14; // 8
// double неявно приводится к int
```

- для сравнения JavaScript:

```
C:\Users\belya>node
Welcome to Node.js v24.15.0.
Type ".help" for more information.
> '5' + 3
'53'
> |
```

Типы данных в Python

Целочисленный int – без ограничений длины

Логический – наследуется от int, True == 1, False == 0

Строковый – неизменяемый

```
x = 10          # int
x = "hello"     # str
x = 3.14        # float
y = x == x

print(y, type(y))
# True <class 'bool'>
```

Тип	C	Python	Java
Целый	int (4 байта)	int (бесконечный)	int (4 байта), long (8)
Вещественный	float, double	float (64 бита)	float (32), double (64)
Логический	_Bool (1 байт)	bool	boolean
Символ	char (1 байт)	str (символ)	char (2 байта, UTF-16)
Строка	char* (массив)	str (объект)	String (объект)

ТОЛЬКО
ОСНОВНЫЕ

float в Python

используют для научных или инженерных расчётов

```
x, y = 1, 2
res = 1 / 2
print(type(res), res)
# <class 'float'> 0.5
```

```
from math import pi
# pi = 3.1415926535...
```

```
radius = 5.0
area = pi * (radius ** 2)
print(area) # 78.53981633974483
print(type(area), f"{area:0.5f}")
# <class 'float'> 78.53982
```

```
# можно задавать проценты или вероятности
probability = 0.85
total_score = 89.5
```

```
# преобразование из строки, файла, ввода пользователя
price_str = "1999.99"
price = float(price_str) * 1.2 # НДС 20%
```

```
# проблема точности
result = 0.1 + 0.2
print(result) # 0.30000000000000004 != 0.3
print(result == 0.3) # False
```

```
# сравнение с погрешностью (эпсилон)
eps = 1e-9 # зададим требуемую точность
if abs((0.1 + 0.2) - 0.3) < eps:
    print("Числа равны с учётом погрешности")
```

bool в Python

1

```
import sys

print(sys.getsizeof(1024))    # 28 байт
print(sys.getsizeof(True))   # 28 байт
print(sys.getsizeof(False))  # 28 байт
print(sys.getsizeof(1))      # 28 байт - как int
```

```
print(1 == True)    # True
print(0 == True)    # False
print(0 == False)   # True
print(9 == True)    # False

print(1 is True)     # False разные объекты
print(0 is False)    # False разные объекты

a, b = True, True
print(a is b)        # True один объект
print(id(True))      # 140724719979264
print(id(a))          # 140724719979264
print(id(b))          # 140724719979264
```

bool — подкласс int, наследует всё от целых чисел
True == 1, False == 0
в памяти всегда только один объект True и один False
занимает 28 байт как маленькое целое число
используется в условных выражениях, циклах, сравнениях

```
value = True

print(type(value))    # <class 'bool'>

print(isinstance(value, bool))
# True - объект класса
print(isinstance(value, int))
# True - bool <= int

print(issubclass(bool, int))    # True
# логическое значение хранится как 0 или 1
```

- любой объект пользовательского типа
- по умолчанию истинный
 - это может быть изменено при наличии в классе `__bool__()`

```
from math import hypot

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def dist(self):
        return hypot(self.x, self.y)

    def __str__(self):
        return f"x = {self.x}, y = {self.y}"

    def __bool__(self): # переопределим bool
        return True if self.x != self.y else False

p1 = Point(3, 4)
print(p1, bool(p1)) # x = 3, y = 4 True
p2 = Point(0, 4)
print(p2, bool(p2)) # x = 0, y = 4 True
p3 = Point(0, 0)
print(p3, bool(p3)) # x = 0, y = 0 False
p4 = Point(4, 4)
print(p4, bool(p4)) # x = 0, y = 0 False
```

bool в Python

2

во встроенных объектах/классах
определён метод `__bool__()`

```
b = 1 # далее три раза True
if b: print(b == 1)
if bool(b): print(b == 1)
if b.__bool__(): print(b == True)
```

```
s = ''; print(bool(s)) # False
s = '1'; print(bool(s)) # True
```

```
dct = {}; print(bool(dct)) # False
lst = []; print(bool(lst)) # False
```

```
dct = {"id":101}; print(bool(dct)) # True
lst = [101]; print(bool(lst)) # True
```

int в Python

Вычислить значение функции $f(2026)$,
если функция от n задана так:

- если $n > 1$, то функция возвращает $(n-1)*f(n-1)$
- иначе - возвращает 1

```
import sys

sys.setrecursionlimit(2200)
sys.set_int_max_str_digits(6000)

def f(n):
    if n > 1:
        return (n-1)*f(n-1)
    else:
        return 1

result = f(2026)
print( len(str(result)) )  # 5819
```

int - пример работы с длинной арифметикой

str в Python

```
x = 1024
s = str(x)
```

```
print(s[0], s[-1]) # 1 4
print(s[::-1]) # 4201
```

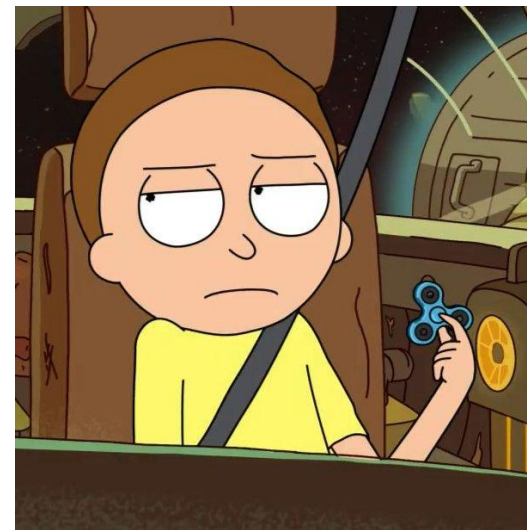
```
# заменить символ в строке
# s[0] = 9 # не работает
```

```
s = '9' + s[1:]
print(s)
```

```
lst = list(s)
lst[0] = '9'
s = ''.join(lst)
print(s)
```

привести примеры работы с символами и строками
показать срезы и неизменяемость строк

можно сравнить с неизменяемыми кортежами
можно сравнить со списками



Дополнительно

некоторые встроенные типы данных

bytes	- неизменяемая последовательность байтов
bytearray	- изменяемая последовательность байтов
list	- список (как динамический массив)
tuple	- кортеж (неизменяемый список)
range	- диапазон
set	- множество
dict	- словарь
function	- функция
module	- модуль
type	- тип

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
print(type(Point)) # <class 'type'>
```

```
def func(): return ...
print(type(func)) # <class 'function'>
```

```
import sys
print(type(sys)) # <class 'module'>
```

Дополнительно

последовательности байтов

```
s = "строка"
```

```
b = bytes(s, "utf-8")
```

```
print(type(b)) # <class 'bytes'>
```

```
# длина в символах и в байтах
```

```
print(len(s), len(b)) # 6 12
```

```
print(b)
```

```
# b'\xd1\x81\xd1\x82\xd1\x80  
  \xd0\xbe\xd0\xba\xd0\xb0'
```

```
b = bytes(s, "cp1251")
```

```
print(type(b)) # <class 'bytes'>
```

```
print(len(s), len(b)) # 6 6
```

```
print(b) # b'\xf1\xf2\xf0\xee\xea\xe0'
```

```
# b[1] = 238 # неизменяемый тип
```

```
print(b.decode('cp1251')) # строка
```

```
b = bytearray("строка", "cp1251")  
print(type(b)) # <class 'bytearray'>  
print(len(s), len(b)) # 6 6  
print(b) # b'\xf1\xf2\xf0\xee\xea\xe0'  
b[1] = 238 # изменяемый тип  
print(b.decode('cp1251')) # сорока
```

```
for ind, elm in enumerate(b):  
    print(ind, elm)  
# 0 241  
# 1 242  
# 2 240  
# 3 238  
# 4 234  
# 5 224
```

Контрольные вопросы

Понятие типа данных

Явная и неявная типизация

Сильная и слабая типизация

Особенности типа в Python:

`int`

`float`

`bool`

`str (char)`



Теоретические основы информатики

Раздел 3. Модели и структуры данных

Тема 5: Типы данных

2026

Беляков Андрей Юрьевич